

Policy assistant evaluation and cost report: Harbor Goods

Contents

Summary	1
1. Scope and method	2
2. Retrieval and the chunking decision	2
3. Answer pipeline gates	3
4. Guardrail evidence	3
5. Cost per question	4
6. Vector store options	4
7. Risks and next steps	5
8. How to reproduce	5

Lab. Harbor Goods and all data here are fictional. Each repository in this portfolio is a separate engagement with Harbor Goods, a fictional mid-size retailer. Account IDs are AWS documentation examples.

Summary

Harbor Goods' store-support team answers staff questions about returns, warranty, shipping and supplier policies by searching documents by hand. This engagement delivers a private question-answering API over those policies: `POST /ask` returns an answer, the policy documents it cites, and the guardrail's verdict. It runs on Amazon Bedrock Knowledge Bases with Amazon S3 Vectors, answers with Amazon Nova Lite through an application inference profile, and enforces a Bedrock guardrail on every answer. The whole stack is Terraform, and every number below comes from code in this repository that `make verify` runs without an AWS account.

Results on the 40-question golden set, offline:

- **Retrieval:** the relevant policy is in the top five results for nearly every answerable question and is usually ranked first (recall@5 and MRR in section 3).
- **No invented policies:** all six questions the documents cannot answer are refused, without a model call for most of them. An answer without a valid citation is replaced by a refusal in code, not by prompt instruction.
- **Cost:** about one US dollar per 1,000 questions with Nova Lite. The guardrail, not the model, is the largest per-question item. The fixed floor is about USD 50 a month, most of it the three private endpoints.
- **Vector store:** S3 Vectors costs cents a month for this corpus; the OpenSearch Serverless floor starts at about USD 175 a month.

What is still open: the offline retriever is a stand-in for Titan embeddings, so the relevance cut-off (`min_score`) and the answer-quality gate must be calibrated with the live run (`make test-live`, section 7) before go-live.

1. Scope and method

- **In scope:** retrieval over 33 policy documents (the fixture corpus in `data/corpus/`, written for this engagement in the style of Harbor Goods’ policies), the answering function, the guardrail policy, the infrastructure, and the cost model.
- **Golden set:** `data/golden.jsonl`, 40 questions phrased the way store staff ask them: 34 answerable, each with the documents that answer it and the facts a correct answer contains, and 6 the corpus cannot answer.
- **Offline retrieval:** `src/harbor_eval/` re-implements the knowledge base’s chunking strategies and scores results with BM25 plus cosine similarity over embedding fixtures (`data/fixtures/embeddings.json`). The fixture vectors come from a deterministic hashed-feature embedder, not from Titan: the offline numbers compare strategies and guard against regressions, and the live run replaces them with real embeddings through the same harness (`--live`).
- **Answer pipeline:** the production code (`src/harbor_rag/`) runs end to end over the golden set with an offline port. Retrieval comes from the local retriever; answers come from an extractive stand-in that quotes and cites the best-matching sentence. That measures selection, citation mapping and refusals, not writing quality.
- **Metrics:** recall@5 and MRR at document level; citation precision is the share of the sources the function shows the model that are relevant; “answered with a relevant citation” and “refused unanswerable” are measured on the function’s output. Thresholds are in `data/eval.yaml` and ADR 0004.

2. Retrieval and the chunking decision

Hybrid retriever, eight results per question, same selection settings for every strategy:

Chunking	Chunks	Recall@5	MRR	Citation precision	Unanswerable with context
fixed-300: Bedrock default (300 tokens, 20 percent overlap)	33	0.971	0.849	0.740	1 of 6
fixed-200: medium fixed chunks	33	0.971	0.849	0.740	1 of 6
fixed-100 (chosen): small fixed chunks	50	0.971	0.864	0.750	1 of 6
hierarchical-300-60: search 60-token children, return parent	73	0.941	0.878	0.760	2 of 6
none: one chunk per document	33	0.971	0.849	0.740	1 of 6

The documents are short (most under 200 words), so fixed-300, fixed-200 and one chunk per document produce the same chunks and the same scores. Fixed-size chunks of about 100 tokens keep recall and raise MRR: the answer sits in a smaller chunk that outranks neighboring policies. Hierarchical chunking (search 60-token children, return the 300-token parent) has the best MRR but misses one document in the top five, and on S3 Vectors its parent text is stored as non-filterable metadata, which counts against the per-vector metadata limit. **Decision: fixed-size, 100 tokens, 20 percent overlap** (ADR 0006); the live run re-checks it with Titan embeddings, where one Bedrock token is shorter than one word, so real chunks hold about 75 words.

The same comparison by retriever, for the chosen chunking. The knowledge base on S3 Vectors searches by vector only, so the vector row is the closest offline proxy for production:

Retriever (chosen chunking)	Recall@5	MRR	Citation precision
hybrid	0.971	0.864	0.750
lexical	0.971	0.863	0.784
vector	0.926	0.790	0.578

3. Answer pipeline gates

Gate	Offline score	Threshold	Result
Recall@5	0.971	0.9	pass
MRR	0.864	0.8	pass
Citation precision	0.750	0.7	pass
Answered with a relevant citation	0.853	0.8	pass
Refused unanswerable questions	1.000	0.8	pass
Answer contains the expected facts	0.471 (stand-in model)	0.8 (live only)	not gated offline

`make verify` fails when any gated score drops below its threshold. Unanswerable questions are refused in two places: most never reach the model, because no chunk clears the relevance cut-off, and the rest are declined by the model and turned into the standard refusal. “Contains the expected facts” is low offline because the stand-in quotes one sentence; it is gated only in the live run, where Nova Lite writes the answer.

4. Guardrail evidence

The guardrail policy is one file, `config/guardrail.yaml`, which Terraform turns into the Bedrock guardrail and the function turns into its local PII pre-filter.

Policy	Setting	Why
Denied topics	legal advice, competitor pricing, employee records	Staff must escalate these (policy ST-05), not ask the assistant
Content filters	hate, insults, sexual, violence, misconduct at HIGH; prompt attack at HIGH on input	Standard for an internal tool that quotes its sources
PII, masked	email, phone, name, address, Harbor Rewards loyalty number	Staff paste customer details into questions
PII, blocked	card, Social Security and bank account numbers	These must never be stored in logs (policy ST-04)
Contextual grounding	grounding 0.75, relevance 0.5	An answer the sources do not support is withheld

Offline evidence (`tests/test_guardrail_policy.py`, `tests/test_pii.py`, mocked `terraform test`):

- every denied topic meets the Bedrock limits and has examples and a red-team case;
- every managed PII type has a pre-filter pattern or a written reason why only the guardrail handles it (names and addresses);
- all 18 red-team cases in `data/fixtures/guardrail_cases.jsonl` get exactly the expected pre-filter result, with no false positives on order numbers, SKUs or prices;
- the deployed guardrail is built from the same file, card numbers are blocked, the prompt-attack filter is on, the grounding threshold is at least 0.7, and the function can only run a published version, never the draft.

The pre-filter exists because the guardrail only sees what goes through **Converse**: the question also goes to **Retrieve**, so it is redacted first, and citation excerpts are redacted the same way (ADR 0005). The logs never hold the question text, only a hash, its length and the PII types found. The live run sends the 18 red-team cases through **ApplyGuardrail** in the block shape of the production request and fails on any unexpected verdict.

5. Cost per question

Mean prompt size over the 35 questions that reached the model: **307 input tokens**. Answer length assumed: **150 output tokens**.

Per 1,000 questions, USD, **us-east-1** on-demand prices pinned in **data/prices.yaml**:

Model	Model	Guardrail	Embedding, retrieval	Lambda, API, logs	Per 1,000 questions
amazon.nova-micro-v1:0	0.03	1.00	0.00	0.03	1.06
amazon.nova-lite-v1:0	0.05	1.00	0.00	0.03	1.09
anthropic.claude-haiku-4.0	1.05 20251001-v1:0.00	0.00	0.00	0.03	2.09

With Nova Lite, the guardrail costs many times more than the model: each question is assessed three times (question, answer, grounding), each at a minimum of one text unit. The function already avoids the largest waste: a question with no relevant context is refused before any model or guardrail call. Beyond that, keep answers short, and move to Claude Haiku only if the live answer-quality gate needs it (about double the per-question cost). Nova Micro saves little because the model is already a small share.

Monthly totals with Nova Lite, including the fixed costs:

Questions per month	Per-question costs	Fixed costs	Total per month
5,000	5.44	50.30	55.74
20,000	21.74	50.30	72.05
100,000	108.72	50.30	159.02

Fixed costs: VPC interface endpoints USD 43.80; KMS keys USD 3.00; CloudWatch dashboard and alarms USD 3.50; S3 Vectors storage USD 0.00.

6. Vector store options

Monthly cost of the vector store alone for the production corpus (about 200 documents, 6,000 chunks, 1,024 dimensions), storage plus the minimum capacity each option bills:

Vector store	5,000 questions	20,000 questions	100,000 questions
S3 Vectors	0.01	0.05	0.25
OpenSearch Serverless (dev/test, 1 OCU)	175.20	175.20	175.20
OpenSearch Serverless (production, 2 OCU)	350.40	350.40	350.40
Aurora PostgreSQL Serverless v2 with pgvector (0.5 ACU)	43.80	43.80	43.80

S3 Vectors has no capacity floor and bills per query, which fits a few thousand staff questions a day. Its trade-offs: query latency in the hundreds of milliseconds rather than tens, no hybrid (keyword plus vector) search in the knowledge base, and a per-vector metadata limit. At this volume none of them is binding (ADR 0002).

7. Risks and next steps

Risk	Likelihood	Impact	Mitigation in this delivery	Next step
Relevance cut-off tuned on proxy scores	High	Too many or too few refusals	<code>min_score</code> and <code>relative_floor</code> are Terraform variables; the live run prints every score	Calibrate with <code>make test-live</code> , record the live gate results here
A policy changes and the index lags	Medium	Answers quote the old policy	Upload triggers ingestion; failed triggers alarm through a dead-letter queue	Add a nightly full sync if uploads are batched
Guardrail false positives on real questions	Medium	Staff get refusals	Red-team set includes clean look-alikes (order numbers, SKUs, prices)	Review interventions on the dashboard for the first two weeks
Staff paste PII the patterns miss (names, addresses)	Medium	PII in the Retrieve query and logs	Guardrail screens the question and masks names and addresses in the answer; logs keep a hash of the question, never its text	Add patterns from the first month of intervention findings
Cost growth from volume	Low	Budget alerts	Reserved concurrency, stage throttling, a monthly budget with forecast alert	Revisit the model choice at 100,000 questions a month

8. How to reproduce

```
make verify      # every offline check; ends with "verify: all checks passed"
make eval        # the retrieval and answer evaluation with the full table
make cost        # the cost model
make report-data # rewrite the generated tables in this report
```

The live run (`make test-live`, maintainer's sandbox account only) deploys the three stacks, ingests the fixture corpus, runs the same evaluation with `--live`, sends the red-team set through the guardrail, calls `POST /ask`, and destroys everything. See `docs/live-test.md`.